

Práctica 6 - ICP y Planeamiento de Trayectorias

I-402 - Principios de la Robótica Autónoma

Prof. Ignacio Mas, Tadeo Casiraghi y Bautista Chasco

Modo de entrega: Enviar por el Aula Virtual del Campus **en un solo archivo comprimido** el código comentado y los gráficos (.jpg ó .pdf) y/o animaciones.

Planeamiento de caminos

Los algoritmos de búsqueda en grafos como Dijkstra o A* pueden ser usados para planear caminos en grafos desde un lugar de inicio hasta un objetivo. Si las celdas de un mapa de grilla se representan como nodos conectados con sus celdas vecinas, estos algoritmos pueden aplicarse directamente para realizar planeamiento para robots. Para este ejercicio, consideramos las 8 celdas vecinas de una celda $\langle x, y \rangle$, que se definen como las celdas adyacentes a $\langle x, y \rangle$ horizontalmente, verticalmente y en diagonal.

El archivo incluido contiene una implementación de planeamiento en 2-D basado en grafos. El script `planning_framework.py` contiene el algoritmo y es el que debe ejecutarse. Los ejercicios de esta sección se realizan implementando las funciones vacías o incompletas que acompañan al script principal.

1. Campos Potenciales Artificiales (APF)

Los campos potenciales modelan el entorno del robot como un sistema de fuerzas virtuales provenientes de la física. El objetivo ejerce una fuerza atractiva, mientras que los obstáculos del mapa generan fuerzas repulsivas.

1. Implementar la función `get_attractive_force` que calcule la fuerza hacia el objetivo utilizando un modelo lineal o cuadrático.

2. Implementar la función `get_repulsive_force`. Esta fuerza solo debe actuar si la distancia al obstáculo más cercano es menor que una distancia de influencia crítica d_0 .
3. Explicar conceptualmente el problema de los **mínimos locales** en este enfoque. Evalúe si esto ocurre para su implementación de campos potenciales.

2. Algoritmo de Dijkstra

El algoritmo de Dijkstra se usa para calcular caminos de costo mínimo dentro de un grafo. Durante la búsqueda, siempre se busca el nodo del grafo con el menor costo desde el punto de inicio y se agregan los nodos vecinos al grafo de búsqueda.

1. Sea $M(x, y)$ un mapa de grilla de ocupación. Durante la búsqueda, las celdas se conectan con sus celdas vecinas para construir el grafo de búsqueda. Completar la función `get_neighborhood` provista que define los vecinos de una celda. La función toma como entrada las coordenadas de una celda y el tamaño del mapa, y devuelve un vector de $n \times 2$ con las coordenadas de sus celdas vecinas, teniendo en cuenta los límites del mapa.
2. Implementar una función para los costos de un arco entre nodos que permita planear caminos de mínima longitud y libre de colisiones. Considerar la celda como un obstáculo si su probabilidad de ocupación supera cierto umbral. ¿Qué umbral se debería elegir? Implementar la función `edge_cost`.
3. Incluir información de ocupación en la función de costo que permita que el algoritmo elija celdas con baja probabilidad de ocupación sobre celdas con mayor probabilidad de ocupación.
4. Implementar el paso de actualización de costos. Para cada nodo *parent*, actualizar el costo de sus *childs* si el costo es menor a través de dicho *parent*.

3. Algoritmo A^*

El algoritmo A^* utiliza una heurística para realizar una búsqueda informada que resulta ser más eficiente que el algoritmo de Dijkstra.

1. ¿Qué propiedades debe tener dicha heurística para asegurar que A^* es óptimo?
2. Definir una heurística para planeamiento de robots móviles en 2-D. Completar la función `get_heuristic` provista. La función toma como entrada las coordenadas de una celda y del objetivo, y devuelve el costo estimado hasta el objetivo.
3. También complete la lógica de actualización como hizo en el punto 2.4

4. ¿Qué pasa si se aumenta la heurística usando h_2 , siendo h_2 un múltiplo de la heurística h definida en el punto anterior. Analizar el comportamiento con diferentes factores: $h_2 = \{1; 2; 5; 10\} * h$

4. Planificación de Cualquier Ángulo (Any-Angle): Theta*

Las grillas tradicionales restringen los movimientos a múltiplos de 45° . Theta* rompe esta limitación permitiendo que el padre de un nodo sea su “abuelo” si existe una línea de visión directa.

1. Utilizarán la función provista `bresenham_line_of_sight` para evaluar la visibilidad entre celdas no adyacentes. Explique brevemente cómo funciona el algoritmo de Bresenham y por qué es eficiente para entornos cuadriculados.
2. Implementar la actualización de costos en `run_thetastar` evaluando las dos condiciones del algoritmo: el camino directo desde el abuelo (salto angular) frente al camino clásico de A^* .
3. Comparar visual y numéricamente la longitud real del camino final obtenido por A^* frente a Theta*. ¿Por qué Theta* logra trayectorias más cortas sobre la misma grilla?

5. Planificación Probabilística: RRT

Los árboles aleatorios de exploración rápida (RRT) evitan la discretización rígida del espacio muestreando posiciones aleatorias continuas en el espacio de configuración libre.

1. Completar la función de extensión (`steer`) dentro de `run_rrt` que toma el nodo más cercano del árbol hacia la muestra aleatoria y genera un nuevo nodo a una distancia máxima fija dada por `step_size`.
2. Analice repetidas veces el resultado de su implementación. ¿Siempre se llega al objetivo en la misma cantidad de pasos? ¿Ante que puede tener dificultad este algoritmo en este mapa?
3. Explicar por qué los caminos generados por RRT clásico poseen una geometría errática (en “zigzag”) y por qué no se garantiza la optimalidad del camino aunque el número de muestras tienda a infinito.

6. Planificación Probabilística Óptima: RRT*

RRT* introduce modificaciones clave para resolver la falta de optimalidad de RRT clásico mediante reestructuraciones locales del árbol.

1. Implementar el paso **Choose Parent**: al añadir un nodo q_{new} , evaluar a todos los vecinos dentro de un radio r para conectarlo al padre que minimice el costo acumulado total $g(q_{new})$ desde el inicio.
2. Implementar el paso **Rewire** (Re-cableado): comprobar si los nodos vecinos dentro del radio r reducen su costo acumulado actual si pasaran a ser hijos de q_{new} . De ser así, modificar sus punteros de filiación.
3. Al ejecutar `run_rrt_star`, notarás que el algoritmo no se detiene al tocar la meta por primera vez. Explicar el concepto de algoritmo *any-time* y analizar cómo evoluciona el costo del camino a medida que aumenta el número de iteraciones.